

Tutorial on Single-Cell Data Analysis

Part 1

Andrea Sottosanti

CIBB 2026 - Rome



UNIVERSITÀ DI PADOVA

Dipartimento
di Scienze statistiche

- ① From biology to data matrices
- ② Data preparation and normalization
- ③ Visualization and dimensionality reduction
- ④ Clustering

Acknowledgments

Part of these slides is based on material provided by Davide Risso for the course “Computational Biostatistics and Bioinformatics”, taught in the Master’s degree programme in Statistical Sciences at the University of Padova.

These slides were prepared with the help of a robot (please spare me when you take over the world).

Who am I?

- I am an Assistant Professor of Statistics at the Department of Statistical Sciences, University of Padova.
- My research focuses on developing clustering and classification methods for spatially and temporally dependent data, with particular interest in applications to genomics, epidemiology, and astronomy.
- I am the course coordinator for “Computational Biostatistics and Bioinformatics’ ’, taught in the Master’s degree programme in Statistical Sciences at the University of Padova.

email: `andrea.sottosanti@unipd.it`

website: `andreasottosanti.github.io`

- Orchestrating Single Cell Analysis (eBook)
 - <https://osca.bioconductor.org>
- scrna-tools.org
- Seurat
 - <https://satijalab.org/seurat/>

The modern biostatistician



I'll run my complex model, and the results will reveal the recipe to cure a new, currently incurable disease.

Expectation

I've just run my complex model, but the biologists said the results don't make any sense



Reality

What to expect from today

What this course **is** meant to do

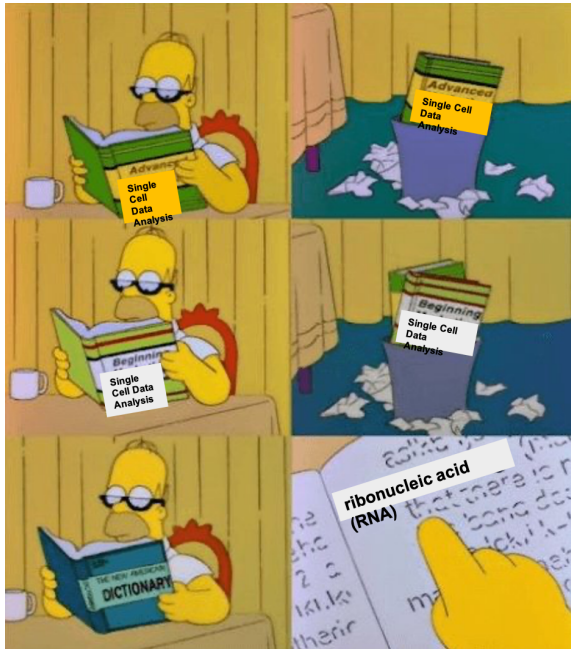
- Provide an introduction to single-cell data analysis and develop some intuition for this type of data.
- Explain why single-cell data require specific statistical and computational tools.
- Build a common language for people coming from different backgrounds:
 - statistics/data science, with little or no biology;
 - biology, with little or no statistics.

What this course **is not** meant to do

- This is not an advanced course on single-cell genomics.
- By 5:30 PM today, you are not going to be an expert in the field (but hopefully you will know what questions to ask and why they matter).

From biology to data matrices

Where it all starts



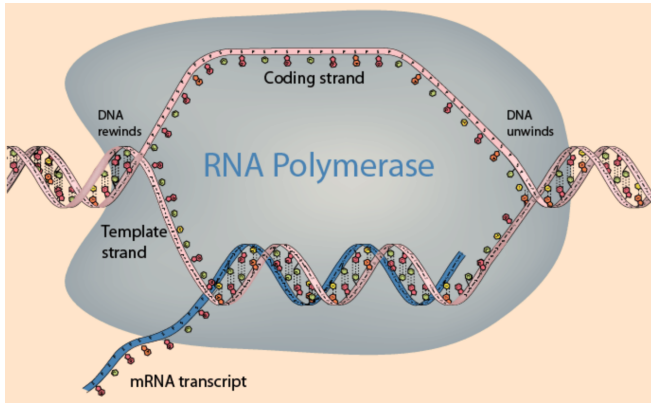
Cells with the same genome can perform very different functions.

Their identity and state depend, in part, on **which genes are expressed**.

- DNA stores genetic information
- Genes are transcribed into RNA
- Messenger RNA (mRNA) can be translated into proteins

The *transcriptome* is the collection of RNA molecules present in a biological sample.

DNA to mRNA



The **expression level of a gene in a cell** can be described as *the number of its mRNA molecules present in the cell at a given time.*

Why is amplification necessary?

We collect a cell and we want to measure its expression levels through a *sequencer*.

To do so, we cannot just “count the number of mRNA molecules”.

In fact, a single cell contains a very small amount of RNA.

The mRNA molecules are converted to artificial molecules of DNA using a retrospective procedure. These molecules are called **complementary DNA** (cDNA).

However, now we just have a small amount of cDNA, and they are still too small to be sequenced directly.

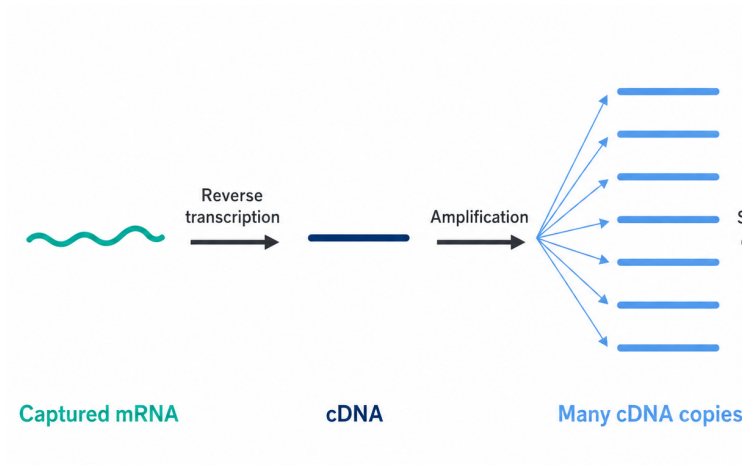
Cell barcode assignment

Single-cell RNA-seq technologies make it possible to:

- ① isolate individual cells from a biological sample;
- ② assign a unique **cell barcode** to each captured cell.

During library preparation, all cDNA molecules derived from the same cell receive the same cell barcode to keep track of the cell they come from.

Data collecting process



Amplification and sequencing

The cDNA molecules are amplified, generating many copies of each molecule.

The sequencing instrument reads short portions of the amplified cDNA and returns their nucleotide sequences.

Each sequence produced by the instrument is called a **read**.

From reads to gene counts

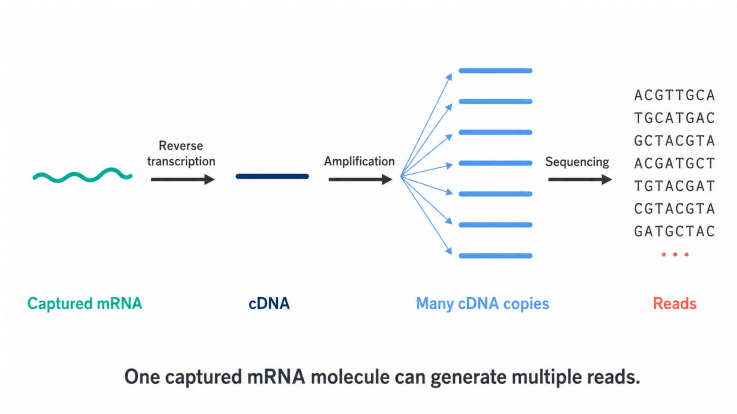
A read is a short sequence of the nucleotide bases A, C, G and T.

Reads are compared with a reference genome or transcriptome to determine the gene from which they most likely originated.

Different reads may correspond to different portions of the same gene.

The number of reads assigned to a gene is called its **read count**.

The whole process



Therefore, with this reverse operation, each gene in each cell is assigned has a certain number of reads, which undirectly reflect the amount of mRNA produced by that gene in a certain period.

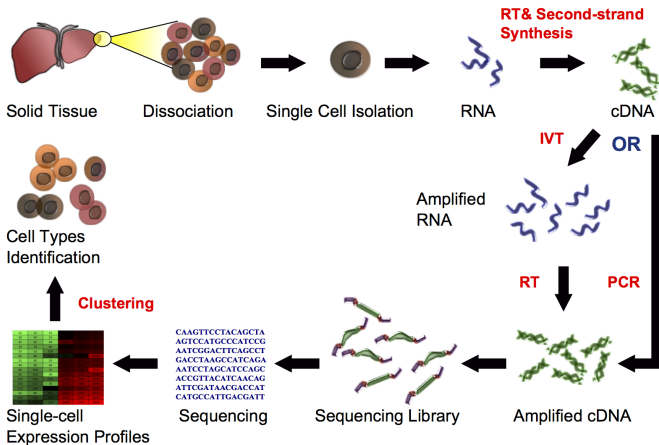
RNA molecules are not observed directly.

The experimental workflow converts them into short nucleotide sequences called **reads**.

- ① RNA is extracted from the biological sample
- ② RNA is converted into complementary DNA (cDNA)
- ③ A sequencing library is prepared
- ④ cDNA fragments are sequenced
- ⑤ Reads are assigned to genes

The number of observed sequences provides information about the abundance of the original RNA molecules.

Single Cell RNA Sequencing Workflow



Challenges in single-cell data analysis

The main issue is that, starting from *very small amounts of RNA*, the RNA needs to be strongly *amplified*. This has two main consequences:

- Some genes, especially those with low expression levels, may fail to be amplified. This results in missing signals and an excessive number of zeros in the data, a phenomenon often referred to as zero inflation.
- Genes that are successfully amplified may instead be amplified disproportionately, resulting in what is known as amplification bias.

These effects lead to an increase in the observed variability of the data.

In addition, sometimes two (or more) cells may occupy the same well or droplet, resulting in so-called doublets: we think we are observing gene expression from a single cell, while in fact the observed expression is the combination of two cells.

Two ways of counting gene expression

After sequencing, multiple reads can be assigned to the same gene.

There are two common ways to construct the count matrix:

Read counts

Every read assigned to a gene is counted.

UMI counts

Some single-cell technologies attach a short molecular label, called a **UMI**, to each captured mRNA molecule before amplification.

Reads carrying the same label are considered copies of the same original molecule and are counted only once.

Read counts and UMI counts

	Read counts	UMI counts
What is counted?	All reads assigned to the gene	Distinct captured mRNA molecules
How?	Every read is counted	Before amplification, each molecule receives a short label called a UMI ; reads with the same label are counted only once
Example	7 reads $\rightarrow$ count = 7	7 reads from 2 labelled molecules $\rightarrow$ count = 2

- read counts measure sequenced copies;
- UMI counts approximate the number of captured mRNA molecules.

Data preparation and normalization

In single cell analysis, we deal with a $n \times p$ matrix, where

- n is the number of cells in the sample;
- p is the number of genes detected.

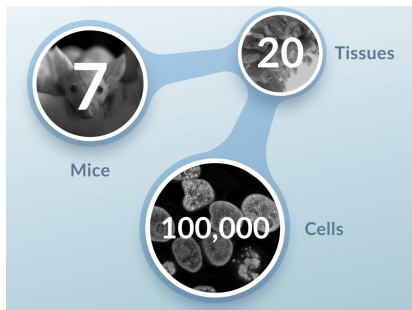
Therefore, Y_{ij} denotes the *read counts in cell i from gene j* , or the *number of UMI counts in cell i from gene j* .

- Y_{ij} large: gene j is highly expressed in cell i ;
- Y_{ij} close to zero: gene j is (almost) unexpressed in cell i ;

It is usually of interest to distinguish different populations of cells based on the *gene expression levels*.

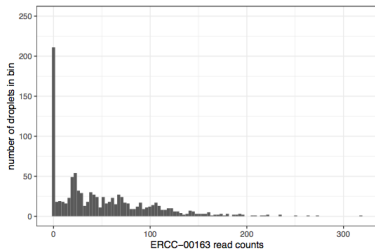
An example: Tabula Muris

Tabula Muris is a project aimed at characterizing mouse cells and consists of almost 100,000 cells from 20 different tissues.

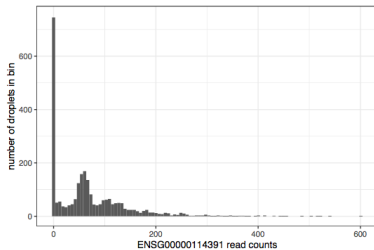


Here, we use a subset of 5,000 cells, considering the 1,000 most variable genes.

Empirical distribution of read counts

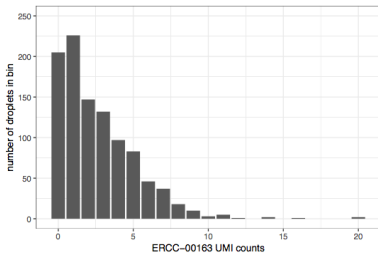


(a) Read counts- technical replicates

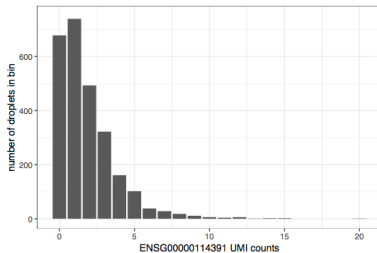


(b) Read counts- biological replicates

Empirical distribution of UMI counts

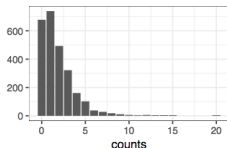


(c) UMI counts- technical replicates

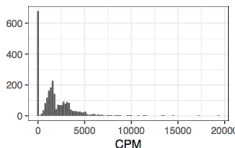


(d) UMI counts- biological replicates

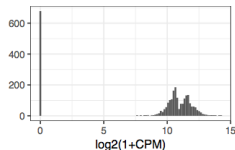
Attention to the logarithm transformation!



(a) UMI counts



(b) counts per million (CPM)

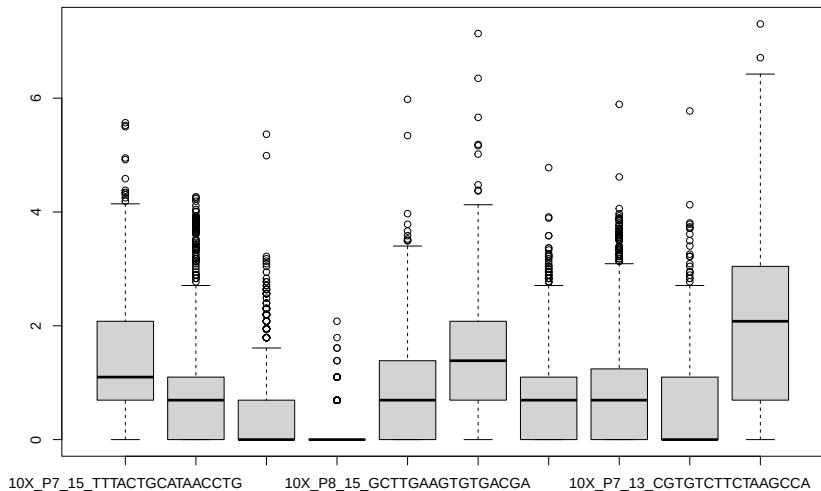


(c) log of CPM

- Non-UMI data: you most likely need to account for the zero inflation
- UMI data: probably not

Tabula Muris: distribution of the reads

First 10 cells



Different cells can have very different **total numbers of observed counts** (row sums).

Distribution of the reads and normalization

This does not necessarily mean that one cell expresses more RNA than another.

Differences can also arise from **technical factors**, such as:

- different amounts of RNA captured from each cell;
- different amplification efficiency;
- different sequencing depth.

Therefore, **raw counts cannot be directly compared across cells**.

Normalization adjusts the data to make gene expression levels more comparable across cells, while trying to preserve the underlying biological differences. Here, we will see the method *scrn*.

Normalizing the gene expression means making the expression levels of the cells comparable among each other.

In practice, one might think that it would be sufficient to divide each cell by the total number of counts:

$$Z_{ij} = Y_{ij} / \sum_k Y_{ik}.$$

However, this is not enough. Consider for instance the measurements of genes A and B in two cells: cell 1, with counts (10, 10), and cell 2, with counts (10, 100). Dividing by the total number of counts for each cell, gene A would result under-expressed in cell 1 compared to cell 2.

To reduce the number of zeros, one can sum the expression profiles of multiple cells and estimate the normalization factors from these pooled counts.

This naturally reduces the number of zeros, but results in a normalization factor for each *pool* of cells rather than for each individual cell.

The solution is to deconvolve the normalization factors estimated for the pools into their individual components, namely the cells.

Main assumption: all cells in the sample have the same expression level (i.e., the genes are not differentially expressed)



The differences arising across cells are *technical*

The scrn algorithm consists of the following steps:

- 1 Create a group of cells (pool).
- 2 For each gene, sum the expression values across all cells in the pool.
- 3 Normalize the pool with respect to a reference.
- 4 Repeat Steps 1-3 for many pools of cells to construct a linear system.
- 5 Deconvolve the normalization factor of each pool into the factors associated with the individual cells.

Let Y_{ij} denote the count for gene j in cell i , and define its expected value as

$$E[Y_{ij}] = \theta_i \lambda_j,$$

where:

- λ_j is the true expression level of gene j ,
- θ_i is the cell-specific bias.

Defining $t_i = \sum_k Y_{ik}$ the total number of counts of cell i , we obtain

$$E[Z_{ij}] = \theta_i \lambda_j / t_i.$$

Normalization of the pool

For a pool of cells S_k :

- $V_{kj} = \sum_{i \in S_k} Z_{ij}$ is the sum of the prescaled expression values for gene j ;
- U_j is the average prescaled expression of gene j across all N cells.

We define

$$R_{kj} = \frac{V_{kj}}{U_j},$$

which compares the expression of gene j in pool k with the reference.

If gene j is not differentially expressed across cells, λ_j cancels out:

$$E[R_{kj}] \approx \frac{E[V_{kj}]}{E[U_j]} \propto \sum_{i \in S_k} \frac{\theta_i}{t_i}.$$

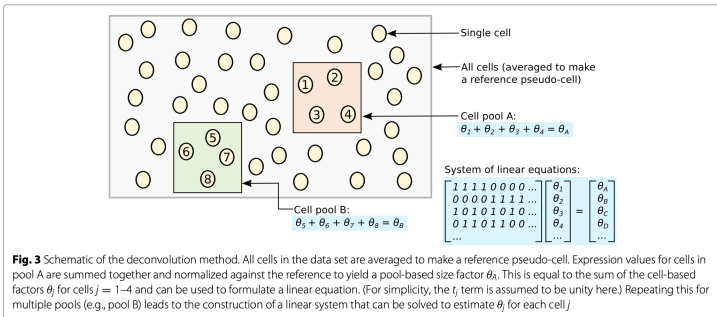
Since the same relationship holds for all non-DE genes, the normalization factor of pool k can be robustly estimated as

$$\hat{r}_k = \text{median}_j \left(\frac{V_{kj}}{U_j} \right).$$

Therefore, for each pool,

$$\hat{r}_k \propto \sum_{i \in S_k} \frac{\theta_i}{t_i}.$$

By creating many overlapping pools of different sizes, we obtain a system of linear equations where the unknown coefficients are θ_i/t_i . These can be estimated via least squares, and the cell-specific normalization factors can then be recovered simply multiplying the estimated coefficients by t_i .



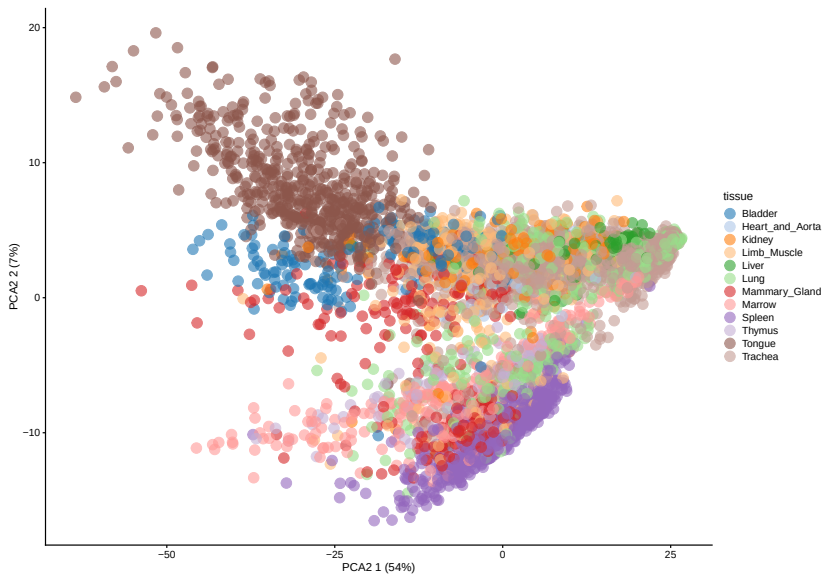
Choice of the pools

Given the assumption of no differential expression, it is important to construct pools using cells that are similar to each other, so that few or no genes are differentially expressed.

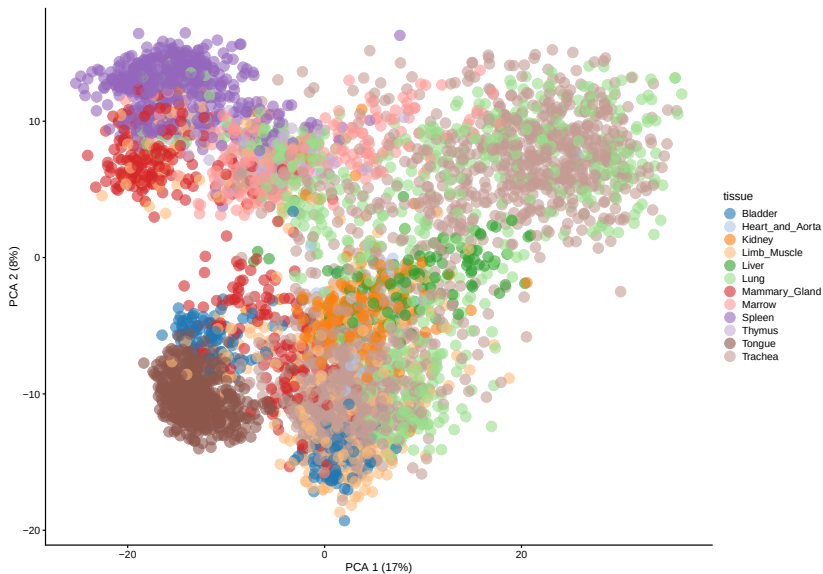
For this reason, when the dataset contains heterogeneous cell populations, a preliminary clustering step is performed before estimating the normalization factors.

The scran algorithm is then applied to each cluster of cells separately.

Tabula Muris: PCA without normalization



Tabula Muris: PCA after normalization



Visualization and dimensionality reduction

Dimensionality reduction

Dimensionality reduction methods are useful for two distinct purposes:

- ① *Visualization* (2-3 dimensions)
 - PCA
 - t-SNE
 - UMAP
- ② *Estimating* a lower-rank signal from a high-dimensional dataset (2 – 50 dimensions)
 - Factor analysis
 - Neural networks (autoencoders)

Visualization is a very important aspect of exploratory analysis of genomic data.

A simple and intuitive approach is to use *dimensionality reduction* methods, such as PCA, to visualize the data in two or three dimensions.

This becomes even more important for single-cell data, which are often characterized by thousands of samples (cells).

Dimensionality reduction

Recall that principal components are *orthogonal linear combinations* of the original variables that maximize the explained variance.

Because of its linearity, PCA is often not sufficient to describe the complexity of the data in only a few dimensions.

Nonlinear methods can better highlight some characteristics of the data, such as their tendency to form clusters.

Dimensionality reduction

There are many nonlinear dimensionality reduction methods. We will consider only one that has become very popular in genomics: *t-distributed Stochastic Neighbor Embedding* (t-SNE).

In the following, we will see what is meant by stochastic embedding and how it can be applied to high-dimensional data.

Note that t-SNE is useful not only for visualizing single-cell RNA-seq data, but also for other high-dimensional datasets, such as those arising in image classification.

Problem description

The problem we want to solve is to find a representation in a two- or three-dimensional space that preserves the similarities between observations in the high-dimensional space.

Stochastic Neighbor Embedding (SNE) is based on the following ideas:

- Distances between points can be represented as conditional probabilities.
- These probabilities should be similar in the original space and in its low-dimensional representation (embedding).
- Neighbors in the original space should remain neighbors in the embedding.

Stochastic embedding (SNE)

The term embedding refers to a low-dimensional representation of the data.

Conceptually, it can be thought of as a “projection” of the data into a lower-dimensional space.

While for some methods this is formally a projection, for nonlinear methods we generally refer to it as an embedding.

t-SNE is a stochastic embedding method: the stochastic component comes from the probabilistic interpretation of the distances between points.

Distances in the original space

Consider two N -dimensional points x_i and x_j . In our case, they represent the expression levels of N genes for cell i and cell j .

The similarity between x_i and x_j is interpreted as the conditional probability $p_{j|i}$ that x_i “selects” x_j as its neighbor, in a model where neighbors are randomly selected proportionally to a Gaussian probability density centered at x_i :

$$p_{j|i} = \frac{\exp\{-||x_i - x_j||^2/2\sigma_i^2\}}{\sum_{k \neq i} \exp\{-||x_i - x_k||^2/2\sigma_i^2\}}.$$

We will see in a few slides how σ_i^2 is estimated.

Since we want a measure of dissimilarity, $p_{i|i}$ is set equal to zero for every i .

Distances in the reduced space

Consider the points corresponding to x_i and x_j in the low-dimensional space, and denote them by y_i and y_j .

In the reduced space, we can similarly compute conditional probabilities:

$$q_{j|i} = \frac{\exp\{-\|y_i - y_j\|^2\}}{\sum_{k \neq i} \exp\{-\|y_i - y_k\|^2\}},$$

where σ^2 is set to 1/2 and $q_{i|i} = 0$.

Distances in the reduced space

If the reduced space is a faithful representation of the original space, we expect $p_{j|i}$ and $q_{j|i}$ to be similar.

Based on this observation, stochastic embedding methods search for a low-dimensional representation of the points that minimizes the difference between $p_{j|i}$ and $q_{j|i}$.

A natural measure of the difference between distributions is the *Kullback-Leibler divergence*. We therefore want to minimize

$$C = \sum_i \sum_j p_{j|i} \log \frac{p_{j|i}}{q_{j|i}}.$$

Properties of the reduced space

Note that the cost function C to be minimized is not symmetric.

Therefore, there is a very high cost when q is low and p is high, i.e., when points that are close in the original space are represented as distant. The cost is much lower when q is high and p is low, i.e., when points that are distant in the original space are represented as close.

As a consequence, these methods preserve the local structure of the data, but not the global structure. We will see how this affects the *interpretation* of t-SNE.

Estimation of σ^2

We still need to specify the variance parameter σ_i^2 .

First, note that it is not appropriate to choose a single σ^2 for the entire dataset, since some regions of the space will contain a higher density of points than others.

In denser regions, a smaller value of σ_i^2 is sufficient, whereas in less dense regions a larger value is appropriate.

The t-SNE algorithm chooses σ_i^2 so that the resulting distribution has a fixed value of “perplexity”, which is a transformation of Shannon entropy.

$$Perp(P_i) = 2^{H(P_i)},$$

where

$$H(P_i) = - \sum_j p_{j|i} \log_2 p_{j|i}.$$

The perplexity value is chosen a priori and can be interpreted as the effective number of neighbors.

The method described so far is called *Stochastic Neighbor Embedding* (SNE). A modification of this method, called *t-SNE*, differs from SNE in the following ways:

- It uses a symmetric version of the dissimilarity;
- It uses a Student's t distribution instead of a Gaussian distribution for distances in the reduced space.

Symmetric cost function

The cost function is modified as follows.

$$C = \sum_i \sum_j p_{ij} \log \frac{p_{ij}}{q_{ij}}.$$

where

$$q_{ij} = \frac{\exp\{-||y_i - y_j||^2\}}{\sum_{k \neq l} \exp\{-||y_l - y_k||^2\}},$$

and

$$p_{ij} = \frac{p_{j|i} + p_{i|j}}{2n}.$$

The main advantage is computational, since the gradient becomes easier to calculate.

Student's t distribution

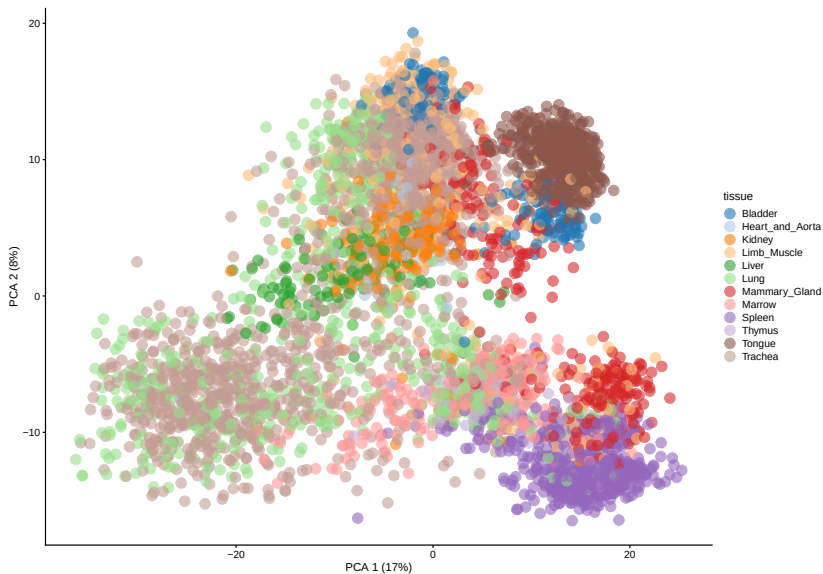
It is useful to use a distribution with heavier tails for the probability distribution in the reduced space, to prevent points at intermediate distances in the original space from being attracted too strongly and appearing too close in the reduced space.

We therefore define

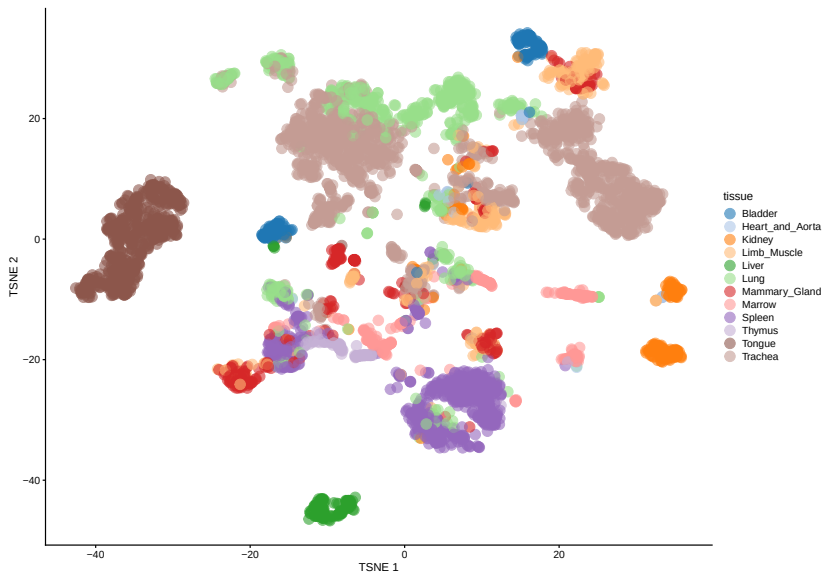
$$q_{ij} = \frac{(1 + \|y_i - y_j\|^2)^{-1}}{\sum_{k \neq l} (1 + \|y_l - y_k\|^2)^{-1}},$$

which corresponds to using a Student's t distribution with one degree of freedom instead of a Gaussian distribution.

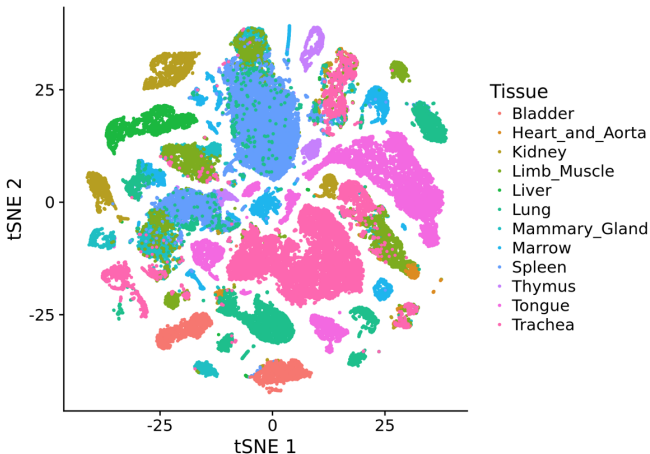
Tabula Muris: PCA



Tabula Muris: t-SNE



Tabula Muris: t-SNE (full dataset)



Problems with t-SNE

Although t-SNE is very useful for visualizing high-dimensional data, great care must be taken when interpreting the resulting plot. In particular:

- Unlike PCA, there is no simple interpretation of the dimensions, and it is therefore not clear how the reduced representation can be used for purposes other than visualization.
- The t-SNE representation preserves the local structure of the data, but not the global structure. It is useful for identifying groups of nearby observations (clusters), but not for interpreting relationships between groups.
- There is no guarantee of convergence to a global minimum, only to a local minimum. Therefore, different initializations of the algorithm may lead to different results.

Factor analysis and PCA

PCA is often introduced as a data transformation technique rather than as a statistical model.

However, there is a relationship between PCA and a class of models called factor analysis models. Given an $n \times J$ matrix, we have

$$Y = W\alpha + \varepsilon,$$

where W is an $n \times k$ matrix of factors, α is a $k \times J$ matrix of “loadings”, and ε is an $n \times J$ matrix of errors.

Factor analysis and PCA

The goal of factor analysis is to find $k \ll J$ factors that describe the original J variables “with the minimum loss of information”.

The first k principal components provide a solution to the factor model.

Indeed, it can be shown that the principal components are obtained by minimizing

$$\min_{i,j} \sum_{i,j} (y_{ij} - w_i^\top \alpha_j)^2,$$

which is equivalent to maximizing the likelihood of a factor model in which we assume that $Y \sim N(W\alpha, \Sigma)$.

Given the previous result, in the case of non-Gaussian data it may be useful to *generalize PCA to other distributions belonging to the exponential family*.

This is the approach proposed by Collins et al. (2002) and applied to single-cell data by Townes et al. (2019).

GLM-PCA (negative binomial)

In particular, for single-cell RNA-seq data we assume that

$$Y_{ij} \sim NB(n_i \exp\{w_i^\top \alpha_j\}, \phi_j)$$

and estimate the parameters w_i , α_j , and ϕ_j by maximizing the likelihood, adding an L2 penalty to stabilize the numerical estimates.

Additional covariates

In RNA-seq data, we often have confounding variables available, such as batch and technical variables describing sample quality.

We would like our factors W not to capture information related to these confounders, so we can include them in the model as covariates.

$$\log \mu = X\beta + (V\gamma)^\top + W\alpha.$$

The ZINB-WaVE model

If the data exhibit zero inflation, the negative binomial distribution may not be sufficient to explain the data, and a zero-inflated negative binomial distribution can be considered.

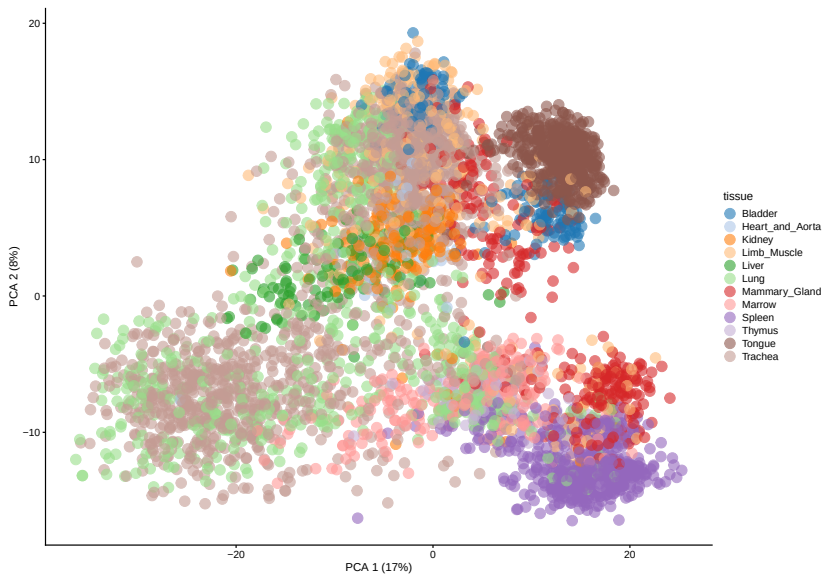
$$f_{ZINB}(y; \mu, \phi, \pi) = \pi \delta_0(y) + (1 - \pi) f_{NB}(y; \mu, \phi), \quad \forall y \in \mathbb{N},$$

As in GLM-PCA, we can consider a model with latent factors and observed covariates, both for the mean of the negative binomial distribution and for the probability of an extra zero.

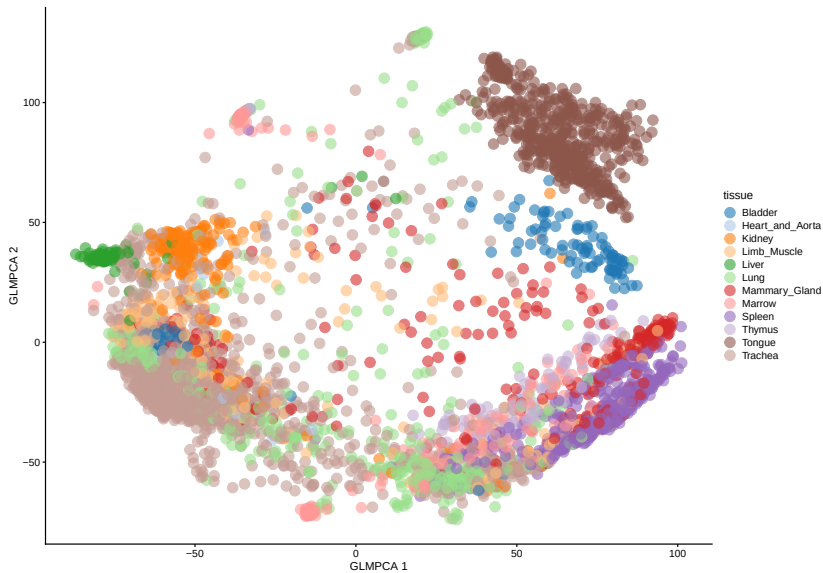
$$\log \mu = X_{\mu} \beta_{\mu} + (V_{\mu} \gamma_{\mu})^{\top} + W \alpha_{\mu}$$

$$\text{logit} \pi = X_{\pi} \beta_{\pi} + (V_{\pi} \gamma_{\pi})^{\top} + W \alpha_{\pi}.$$

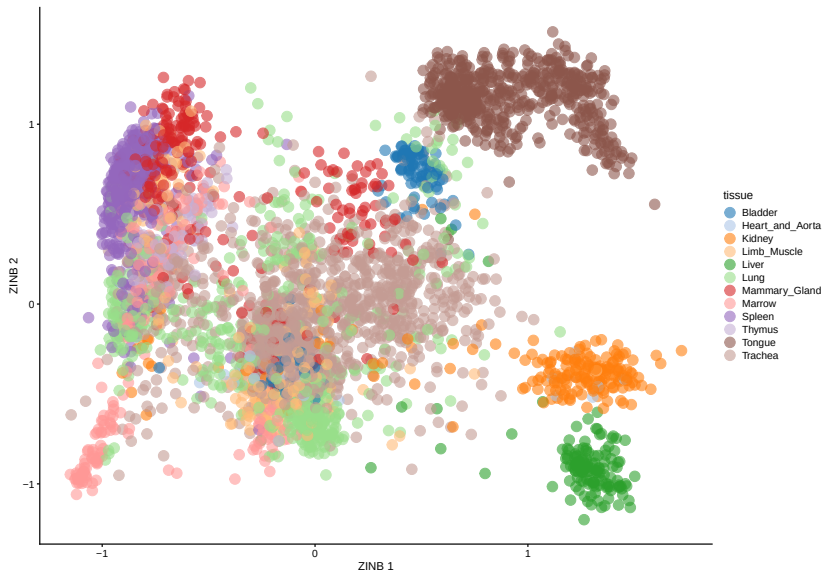
Tabula Muris: PCA



Tabula Muris: GLM-PCA



Tabula Muris: ZINB-WAVE



Advantages and disadvantages

GLM-PCA and ZINB-WAVE have the following advantages:

- They account for the count nature of the data, which is particularly important for single-cell data characterized by low counts.
- Since they are linear methods (apart from the link function), the factors can be interpreted similarly to those in PCA.
- The possibility of including covariates makes it possible to remove batch effects and account for confounding variables.

The disadvantages are:

- Greater computational complexity compared with PCA.
- There is no guarantee of convergence to a global minimum.

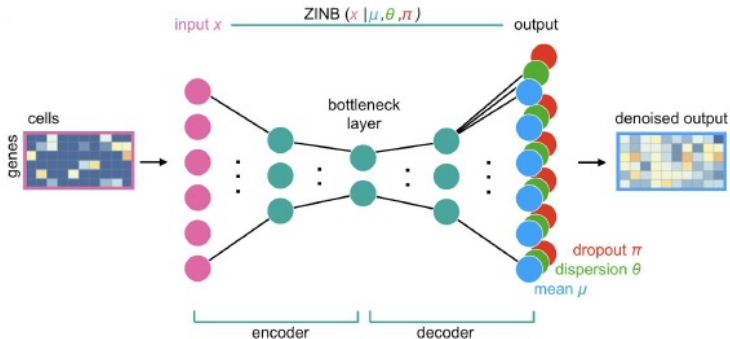
Neural networks (overview)

In recent years, many neural network approaches have been proposed for dimensionality reduction of single-cell data.

We will not go into the details of these models, but they are particular neural networks called *autoencoders*. Rather than solving a classification problem, they aim to describe the original J variables in a lower-dimensional space.

These models can be viewed as nonlinear versions of the factor analysis models we have just discussed.

Autoencoders (overview)



Advantages and disadvantages

Autoencoders have the following advantages:

- Since they are based on neural networks, they can capture complex nonlinear relationships between the latent factors and the original variables.
- Implementations using *keras* and *pytorch* in Python allow the use of GPUs, making them very competitive from a computational point of view.

The disadvantages are:

- The factors are not easy to interpret, and there is a risk of using these models as *black boxes*.
- There is no guarantee of convergence to a global minimum.

- Dimensionality reduction is useful both for visualizing the data and for estimating $K \ll J$ factors that provide a lower-dimensional representation of the data.
- Depending on the objective (inference or visualization), different methods may be preferable.
- In the next lecture, we will see how the reduced space can be used to identify cell populations (clustering) or analyze cellular development (pseudotime).

Clustering

Why do we need clustering?

Single-cell RNA-seq allows us to measure gene expression in thousands of individual cells, but usually **we do not know their identity**.

One of the main goals of the analysis is therefore to identify groups of cells with similar expression profiles.

These groups may correspond to:

- different **cell types**;
- different **cell states**;
- other biologically meaningful populations.

From expression to clusters

Let Y_{ij} denote the normalized expression of gene j in cell i .

The goal is to partition the cells into groups such that cells within the same cluster have similar expression profiles.

In practice, clustering is usually performed in a **lower-dimensional space**, for example using the first principal components.

Genes $\longrightarrow$ PCA $\longrightarrow$ Clustering

One of the simplest clustering algorithms is *K-means*.

We assume that the number of clusters K is known.

The goal is to find K centroids $\mu_1, \dots, \mu_K$ that minimize the within-cluster variability:

$$\sum_{k=1}^K \sum_{i:c_i=k} \|y_i - \mu_k\|^2.$$

Each cell is assigned to its closest centroid.

K-means algorithm

The algorithm iterates two simple steps:

- 1 **Assignment:** assign each cell to its closest centroid:

$$c_i = \arg \min_k ||y_i - \mu_k||^2.$$

- 2 **Update:** recompute each centroid as the average of the cells assigned to that cluster.

These two steps are repeated until the assignments no longer change.

Limitations of K-means

K-means is simple and computationally efficient, but it makes strong implicit assumptions.

In particular, it works best when clusters are:

- approximately **spherical**;
- characterized by similar variability;
- well separated in the selected space.

A more flexible approach is to explicitly model the distribution of the data.

Gaussian mixture models

A Gaussian mixture model assumes that the data arise from K different Gaussian distributions:

$$y_i \sim \sum_{k=1}^K \pi_k \mathcal{N}(\mu_k, \Sigma_k).$$

Each cluster has its own:

- mean μ_k ;
- covariance matrix Σ_k ;
- probability π_k .

Unlike K-means, clusters can therefore have different *shapes*, *orientations* and *sizes*.

The R package `mclust` provides very a efficient implementation to estimate a Gaussian mixture model varying the degree of flexibility of the mixture components

From mixtures to clustering

For each cell, the model estimates the probability of belonging to each cluster:

$$P(z_i = k \mid y_i).$$

The cell can then be assigned to the cluster with the highest probability.

This provides a probabilistic interpretation of clustering and allows more flexible cluster shapes than K-means.

Community-based clustering

For large single-cell datasets, *community-based clustering* is one of the most widely used approaches.

Instead of modeling the complete distribution of the data:

- ① each cell is a node;
- ② cells with similar expression profiles are connected;
- ③ clusters are identified as *communities* of highly connected cells.

The graph is usually constructed in a reduced-dimensional space, such as PCA space.

Building the graph

For each cell, we identify its k nearest neighbors.

This produces a nearest-neighbor graph. Once the graph has been constructed, we search for groups of nodes that are *highly connected within the group* and weakly connected *between groups*.

Popular algorithms include:

- Louvain
- Leiden**
- Walktrap

Louvain and Leiden search for partitions with high **modularity**, i.e. more within-cluster connections than expected by chance.

How good is our clustering?

Without known labels, we can evaluate the internal structure of the clusters using the *Silhouette index*:

$$sil(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}.$$

where

- $a(i)$ = average distance from cells in the same cluster;
- $b(i)$ = average distance from the closest alternative cluster.

Values close to 1 indicate well-separated cells, while negative values suggest possible misclassification.

Comparing with known labels

If an external classification is available, we can compare it with the clustering.

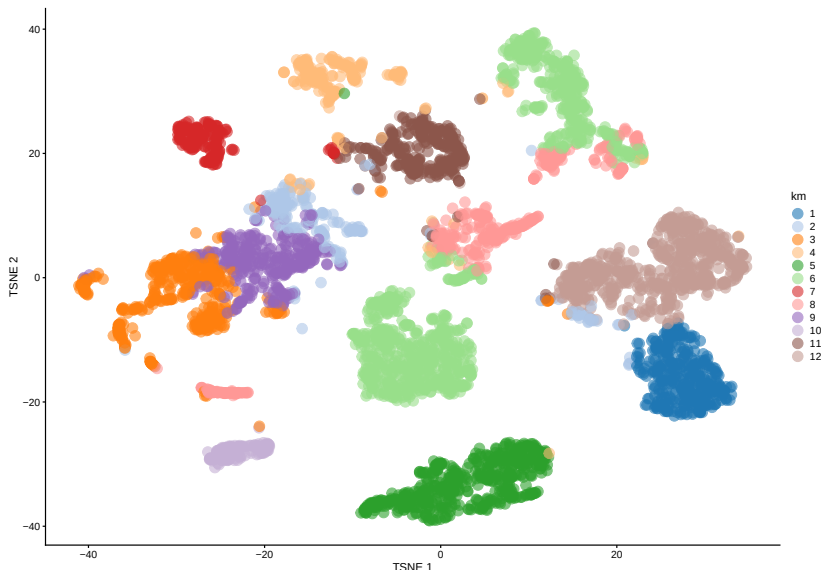
A common measure is the **Adjusted Rand Index (ARI)**.

It measures how often two partitions agree on whether pairs of observations belong to the same cluster, while correcting for agreement occurring by chance.

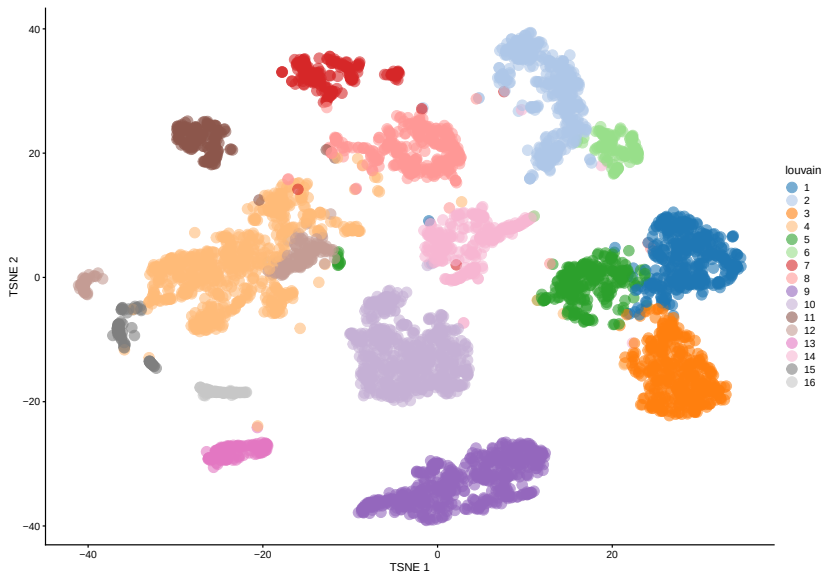
$$ARI \approx \begin{cases} 1 & \text{strong agreement} \\ 0 & \text{agreement expected by chance} \end{cases}$$

Example: Tabula Muris (k-means, $K = 12$)

We cluster 5,000 cells using the first principal components of their expression profiles.



Example: community-based clustering (Louvain)



Last considerations on single-cell clustering

Different clustering methods may identify different structures in the same dataset.

In some datasets, external information is already available, such as the tissue of origin or previously assigned cell-type labels. These annotations can be used to compare the clusters obtained from the expression data.

However, tissue labels are useful mainly as external biological information and do not necessarily correspond to true cell-type labels.

From clusters to cell types

Clustering identifies groups of similar cells, but it does not tell us what these cells are.

A classical approach is to identify *marker genes* that distinguish one cluster from the others.

Alternatively, we can compare the cells with an already annotated reference dataset.

While the clustering methods discover populations, reference-based methods are more thought to give them a biological identity.

Reference-based annotation: SingleR

SingleR uses an annotated reference dataset to assign cell-type labels.

At a high level:

- 1 Compare the expression profile of each cell with the profiles of the reference cell types.
- 2 Compute a similarity score using the Spearman correlation.
- 3 Assign the most compatible cell-type label and repeat Steps 1–2 using increasingly specific labels.

The procedure first identifies the broad cell type and then progressively refines the annotation into more specific subtypes.

For example, a cell may first be classified as a T cell. The analysis is then repeated considering only more specific T-cell subtypes, such as CD4⁺ T cells, CD8⁺ T cells, or regulatory T cells.

Final considerations: differential expression

Suppose we want to test whether gene j is differentially expressed between cell populations A and B .

- If A and B are clusters discovered from the same expression data, their labels are themselves data-driven.
- The same data are therefore used both to define the groups and to test for differences between them.
- As a consequence, standard differential expression tests no longer guarantee control of the nominal Type I error.

Final considerations: differential expression

Two main solutions have been proposed:

- ① *Selective inference*: use statistical tests that explicitly account for the fact that the clusters were selected from the data, controlling the selective Type I error.
- ② *Count splitting*: split the observed counts into two approximately independent datasets, using one for clustering and the other for differential expression testing.

The general principle is to account for, or break, the dependence between **cluster discovery** and **statistical testing**.